

SIH26036 — Final Review Brief

Team SahiDaam | Online Verification System for Weighing and Measuring Instruments | Written 9 September 2026, 23:00
For the team and for our mentor. **Final review: Thursday 10 September, 09:30, PRP Annexe 203.**

1. Where we are, in one paragraph

The system is built, deployed and working. A trader applies, an officer logs in and records the verification, and the system issues a certificate carrying a QR code. Scanning that code returns the certificate's status **on the day of the scan** — VALID, EXPIRING SOON, EXPIRED or NOT FOUND. Since Friday we have added: login with four roles, the issuing officer's name on every certificate, an audit log, a digital signature that breaks if anyone edits a stored field, four automatic fraud checks, per-State configuration, and an enforcement panel that names the section of the Act and the penalty. **43 automated tests pass.** Everything our mentor asked for on 4 September is answered, either in the software or in a sourced document.

The nine questions Sir asked, and where each is answered

Question	Answered by	Where to see it
How GATCs?	Act s.24(3) - notified by the Central or the State Government. PIB: 51 recognised, and a GATC may verify only in its approved State.	fraud.py, rule 2. Shreyash's notes.
How to scale up?	Per-State config files. Three States ship with different fees, reminder windows and daily limits. A fourth State is a new file, not new code.	states/TN.json, DL.json, MZ.json
In WB no verification centre?	Checked. As stated it is not supported - WB publishes 19 district offices and 113 inspection units. See section 6.	research/west_bengal.md
How to handle fake verification?	Every certificate is signed with an Ed25519 key. Edit any stored field and the status page says NOT VERIFIED.	signing.py. Live on the status page.
How to catch fakers?	Four automatic checks at the moment of issue, plus a public form anyone can use to report an instrument.	fraud.py, /report
What actions against law-breakers?	The status page of an expired certificate shows the section, the penalty and whether it can be compounded.	enforcement.json, /verify page
What is the legal procedure?	Inspection and seizure under s.15, the penalty section, then compounding under s.48 or prosecution, appeal under s.50.	enforcement.json
Who are convicted?	Section 49 - the nominated person responsible, or every person in charge at the time, and the company itself.	enforcement.json
How to get accountability?	Named officer on every certificate, an audit log of every action, and roles so only the right person can act.	auth.py, /admin/audit

2. What every file does — open only the one you need

Folder: *Desktop* → *sih internals 2026* → *final working prototype v1*. Nobody needs to read the whole project. Find your row, open that file.

The programs

File	What it does	Open it to...
app1.py	The main program. All the web addresses, the database tables, issuing a certificate, the public status page, the due list.	change how a certificate is made or shown
auth.py	Login, the four roles, the audit log, the officer dashboard, the public report form, and the demo accounts that get recreated on every start.	change anything about who can do what
config.py	Reads the State files and the law file. No settings live in the code.	change how settings are loaded
signing.py	Makes and checks the digital signature on a certificate.	change anything about the signature
fraud.py	The four checks that catch a faker. Plain functions, no database.	add or change a fraud rule
seed_users.py	Old manual way to create accounts. Kept for local use only — auth.py now does this automatically.	create accounts by hand locally

The settings and data

File	What it does	Open it to...
states/TN.json	Tamil Nadu: fees, periods, 60-day reminder window, daily limit 40.	change a Tamil Nadu setting
states/DL.json	Delhi: 45-day reminder window, limit 35, higher fees.	change a Delhi setting
states/MZ.json	Mizoram: 90-day reminder window, limit 15, lower fees.	change a Mizoram setting
enforcement.json	Shreyash's file. For each offence: the section, the penalty, whether it can be compounded, and the officer's next step.	correct a section or a penalty
requirements.txt	The list of libraries the app needs.	add a new library

The screens (templates folder)

File	Which screen
templates/form.html	The application form. Instrument, State, class, maximum permissible error.
templates/certificate.html	The certificate itself, with the QR code.
templates/verify.html	The public status page a QR scan opens. Also shows the signature result and the enforcement panel.
templates/dashboard.html	The officer dashboard: due, expired, fraud alerts, public reports.
templates/reminders.html	The plain due list.
templates/login.html	The login screen.
templates/audit.html	The audit log, admin only.
templates/report.html	The public 'report a suspicious instrument' form.
templates/403.html	The page shown when someone is not allowed in.

The tests and the documents

File	What it is
test_app.py	31 checks on the whole app: logins, roles, expiry, signature, enforcement, fraud, States, public pages.
test_fraud.py	12 checks on the four fraud rules. Two per rule: one where it must stay quiet, one where it must fire.

File	What it is
DEPLOY.md	Why login was failing, Render vs Netlify, and the free database option.
README.md / INTEGRATION.md	Abhay's notes on how the pieces fit together.
research/west_bengal.md	Krishna's West Bengal findings.

3. Getting the changes into GitHub — copy these exactly

Do this **tonight**. The fixes are on your computer but not yet on GitHub, and Render only deploys what is on GitHub. Until you push, the live site still has the old broken login.

Step 1 — open a terminal in the right folder

Open File Explorer, go to *Desktop* → *sih internals 2026* → *final working prototype v1*, click in the white address bar at the top, type **cmd** and press Enter. A black window opens, already in that folder.

Step 2 — the four commands

```
git add .
git commit -m "Fix login blocker, add signing, fraud rules, State config, tests"
git pull --rebase
git push
```

Line 1 gathers every change. Line 2 saves them with a note. Line 3 brings down anything a teammate pushed while you were working, and puts your work on top of theirs. Line 4 uploads. Render then rebuilds by itself, which takes two to four minutes.

If this folder is not a git folder yet

If line 1 says *not a git repository*, run these first:

```
git init
git remote add origin
https://github.com/icebergabhay1111-a11y/legal-metrology-certificate-verification.git
git branch -M main
git fetch origin
git reset --soft origin/main
```

Then run the four commands above. The **reset --soft** line makes git treat your files as changes to the existing project rather than a brand new one, so nothing on GitHub is lost.

Getting other people's changes

```
git pull
```

Run it before you start work and before you push. If it complains about a conflict, stop and call Abhay — do not guess at midnight.

Checking it worked

1. Type `git log --oneline -3`. Your commit should be at the top.
2. Open the repository page on GitHub. Your files should be there, with the time just now.
3. Open the Render dashboard. There should be a deploy in progress or just finished.

4. Running the server and looking inside the database

A. Running it on your own laptop

1. Open the folder and type **cmd** in the address bar, as above.
2. Type `pip install -r requirements.txt` and press Enter. This downloads Flask and the other libraries. It takes about a minute, once.
3. Type `python app1.py` and press Enter.
4. The window prints a web address, normally `http://127.0.0.1:5050`. Open that in your browser.
5. Log in as **lab1** with password **sahidaam2026**.
6. To stop the server, click the black window and press Ctrl and C together.

B. Running the tests

```
python -m pytest -q
```

It should finish in about twenty seconds and print **43 passed**. If it prints anything else, something is broken and you should not push.

C. Looking inside the database on your laptop

The database is a single file called **certificates.db**, created the first time you run the app. It holds five tables: certificates, users, audit, reports and fraud_alerts.

1. Download **DB Browser for SQLite** from sqlitebrowser.org. It is free and needs no account.
2. Install it, open it, click **Open Database**, and choose **certificates.db** from the project folder.
3. Click the **Browse Data** tab. Pick a table from the dropdown at the top left.
4. Look at the **users** table. The password column shows a long string starting with *script:* or *pbkdf2:* — that is the hash. **No password is stored in readable form**. If a judge asks to see it, show them this.
5. Look at the **audit** table to see who did what and when.
6. Close the program before running the app again, or the file may be locked.

If you would rather not install anything, type this in the project folder:

```
python -c "import sqlite3;print(sqlite3.connect('certificates.db').execute('select id,username,role from users').fetchall())"
```

D. The live server on Render

Live address: **legal-metrology-certificate-verification.onrender.com**

1. Sign in at dashboard.render.com and click the service.
2. **Logs** on the left shows what the server is doing right now. If a page is broken, the reason is here.
3. **Environment** is where settings live. Three matter: **BASE_URL** (the public address, or QR codes point nowhere), **SECRET_KEY** (any long random text), and **SIGNING_KEY** (from `python signing.py --print-key`, so signatures survive a redeploy).
4. **Manual Deploy** → **Deploy latest commit** rebuilds from GitHub.

You cannot open the database file on Render, and this is important. Render's free plan wipes the filesystem on every deploy, restart or sleep — their own words. So certificates you create today disappear on the next deploy. Two consequences. First: after your **last** deploy tonight, create the demonstration certificates again, in order, so CERT-0001 is the one your slide's QR points to. Second: to look inside the live data, use the app's own pages — **/admin/audit** as admin1, and **/dashboard** as an officer. Those are the database, shown through the app.

Render also puts a free service to sleep after 15 minutes with no visitors, and takes about a minute to wake. **Open the site at 09:15 and leave the tab open.**

5. What is left — tonight and tomorrow morning

It is nearly midnight and the review is at 09:30. The list below is short on purpose. Do the four things in Tonight, then stop and sleep. Anything not on this list waits until after the review. Adding a new feature now is how a working system becomes a broken one.

Tonight — about forty minutes

#	Do this	Who
1	Run <code>python -m pytest -q</code> and confirm 43 passed.	Anikeit
2	Push to GitHub with the four commands in section 3. Wait for Render to finish.	Anikeit
3	Set <code>SIGNING_KEY</code> , <code>SECRET_KEY</code> and <code>BASE_URL</code> on Render if they are not already set.	Anikeit
4	On the live site, log in as lab1 and create the three demonstration certificates: a weighbridge dated today, a fuel dispenser dated 335 days ago, and a fuel dispenser dated 400 days ago. Then scan the first one's QR with your own phone and confirm it says VALID.	Anikeit
5	Send this brief to the team and to Sir.	Anikeit

Tomorrow, 08:30 to 09:20

Time	Do this
08:30	Everyone reads their own speaking script once, out loud, timed.
09:00	Open the live site on the presenting laptop. Open <code>/dashboard</code> , <code>/admin/audit</code> and one <code>/verify</code> page in three tabs. Leave them open so Render does not go to sleep.
09:10	Scan the slide's QR with two different phones. If either fails, use the second phone during the review and say nothing about it.
09:15	Agree who holds the phone, who drives the laptop, and who answers each question. The list is in the Q&A drill in the speaking scripts.
09:30	Review, PRP Annexe 203.

After the review — not before

- Move the database to Neon so data survives a redeploy. Steps are in `DEPLOY.md`.
- Anvita's Figma screens, and matching the built pages to them.
- Read the eMaap Functional Requirement Specification. It is the Department's own statement of what eMaap is meant to do, and none of us has opened it.
- Replace the placeholder fee figures in the three State files with published fee schedules.
- Obtain the gazette text of rule 27.

6. What we will say if asked — the honest list

Every item below is something we know and have chosen to state plainly rather than be caught on. This is for Sir as much as for us.

If asked about	We say
The re-verification periods (24 / 60 / 12 months)	They follow rule 27(2) of the Legal Metrology (General) Rules, 2011. We have confirmed 27(2) is the right provision. We have not read the gazette text. So in our system they are configuration, not hard-coded law, and the certificate prints that they are unverified.
Reminders to the holder	There is no notification to the holder yet. What exists is the officer's due list. We will not claim more. And on other States: the published documentation of Delhi, Tamil Nadu, Maharashtra and Karnataka describes no reminder before expiry — that is an absence in four documents, not proof that no State sends one.
West Bengal	The claim that WB has no verification centre is not supported: WB publishes 19 district offices, 113 inspection units and two standard laboratories. WB also already runs e-Parimap with a public certificate search, so we do not claim to be the first public lookup. Our claim is narrower: status on the day of the scan, expiry calculated by the system, and what is about to lapse surfaced before it does.
The State fee figures	The three State files carry placeholder fees, marked as such inside the files. The revenue figures we quote (Madhya Pradesh Rs 25.60 crore, Delhi Rs 10.39 crore, Mizoram Rs 17.61 lakh) are real and cited to page. The per-instrument fee schedules are not yet sourced.
The database	SQLite on Render's free tier, which is wiped on redeploy. We know why, we know the fix (a free hosted Postgres), and we have written it up. We did not attempt the migration the night before a review.
Whether the certificate is legally valid	No. It is a record in our system, not a statutory certificate. It is digitally signed, and it records the technical facts a verification establishes. Making it a legal instrument needs a signature under the IT Act and adoption by the department.
What we got wrong	Two figures withdrawn and three legal citations corrected during our own audit. A file we held as the General Rules turned out to be the National Standards Rules. A commercial legal database we relied on has its rule numbers wrong. We found all of it ourselves.

7. Figma

The design file is at figma.com/design/8LAeA5nUvpRq5INiNmP8Lq/sahidaam-rough. It could not be read automatically because Figma's Dev Mode server is switched off. To turn it on: open the Figma **desktop app**, open the file, then Figma menu → Preferences → **Enable Dev Mode MCP Server**, and restart the Claude desktop app. This is worth doing after the review, not tonight.

Last word, for the team. The system works, the tests pass, and every number we will say has a document behind it. The things we do not know are written down and we will say them out loud. That is a stronger position than most teams will be in tomorrow morning. Get some sleep.